

SESIÓN 7

Despliegue Continuo, Ambientes y Rollback

Informe de ejecución de los tres laboratorios

Estudiante	Albert Omar Araya Cory
Repositorio	github.com/albrtaraya/fullstack-lab-1-1ci-cd
Pull Requests	#10, #12 a #16
Proyecto Railway	brave-renewal — ambientes staging y production
Staging	fullstack-lab-1-1ci-cd-staging.up.railway.app
Producción	fullstack-lab-1-1ci-cd-production.up.railway.app
Modalidad	Individual
Fecha	22 de septiembre de 2026

Estado de la sesión

Los tres laboratorios completos · Despliegue real en staging y producción
Incidente provocado, diagnosticado, revertido y corregido por el flujo normal

Resumen de ejecución

La Sesión 7 cierra el ciclo de CI/CD: el pipeline debe desplegar solo a staging tras cada integración, producción debe requerir una decisión manual, y un despliegue defectuoso debe poder revertirse en segundos.

Laboratorio	Nombre	Estado
1	Conectar el pipeline a Railway	Cumplido
2	Ambientes de staging y producción	Cumplido
3	Simulacro de Caos: el Ctrl+Z de Producción	Cumplido

Lo que costó llegar al primer despliegue real

Conectar el pipeline con Railway no salió a la primera. Cada intento fallido dejó un error concreto en el log, y cada uno se resolvió con un Pull Request propio:

Intento	Qué respondió Railway	Causa	Corrección
1	El job se omitía con un aviso	No existía el Secret RAILWAY_TOKEN	Cargar el Secret en GitHub
2	Invalid RAILWAY_TOKEN	El servicio se llama fullstack-lab-1-1ci-cd, no backend; y el token era de workspace	PR #12: nombre real del servicio
3	Unauthorized	Un token de workspace no sirve para railway up, ni como RAILWAY_API_TOKEN	PR #13, descartado después
4	Deploying as astro static site	Railway ya no lee railway.json en servicios nuevos	Configurar el servicio en el panel
5	Healthcheck succeeded! Deploy complete	—	PR #14: un token de proyecto por ambiente

Laboratorio 1 — Conectar el pipeline a Railway

Objetivo: crear el proyecto en Railway a partir del repositorio y hacer que el pipeline dispare el despliegue a staging, pero solo cuando todos los controles anteriores hayan pasado.

Proyecto, base de datos y variables

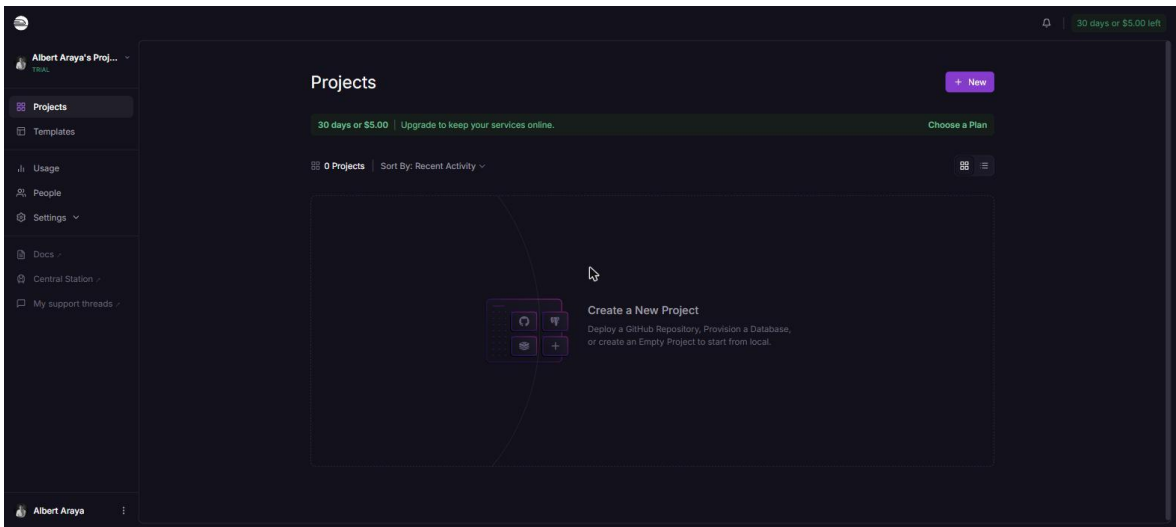


Figura 1. La cuenta de Railway antes de empezar: plan TRIAL con \$5.00 y ningún proyecto.

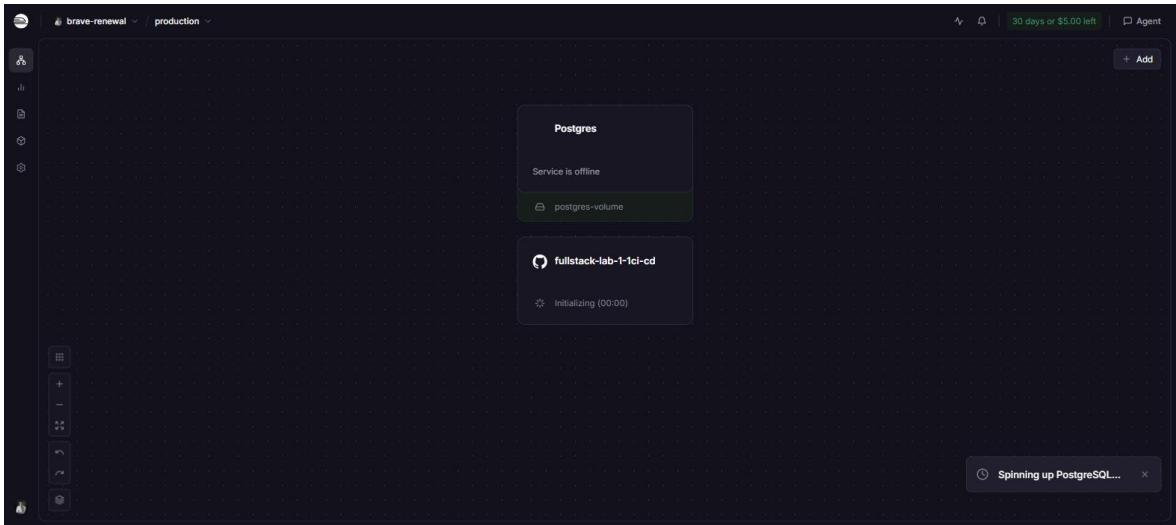


Figura 2. PostgreSQL aprovisionándose junto al servicio, con su volumen para conservar los datos.

Variable	Valor	Por qué
DATABASE_URL	\${Postgres.DATABASE_URL}	Referencia al plugin: si la contraseña rota, la referencia sigue siendo válida
JWT_SECRET	Propio de cada ambiente	En staging lo genera Railway con \${secret(48)}: distinto del de producción y nadie lo ve

PORT	4000	Coincide con EXPOSE y con el HEALTHCHECK del Dockerfile
RAILWAY_DOCKERFILE_PATH	Dockerfile.backend	Le indica a Railway qué imagen construir (ver el hallazgo más abajo)

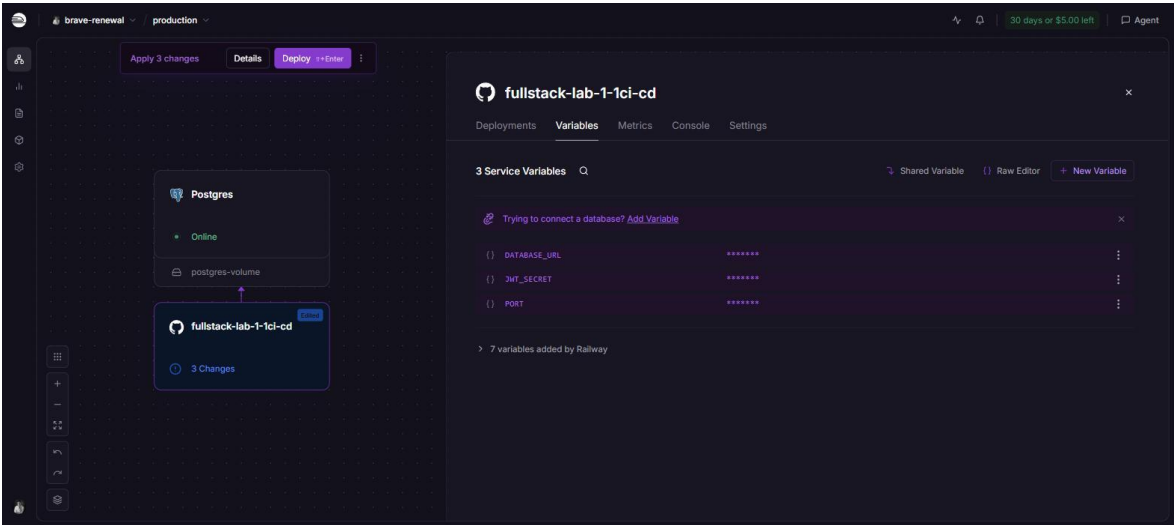


Figura 3. Las variables cargadas y enmascaradas, con la flecha que enlaza el servicio con el plugin de Postgres.

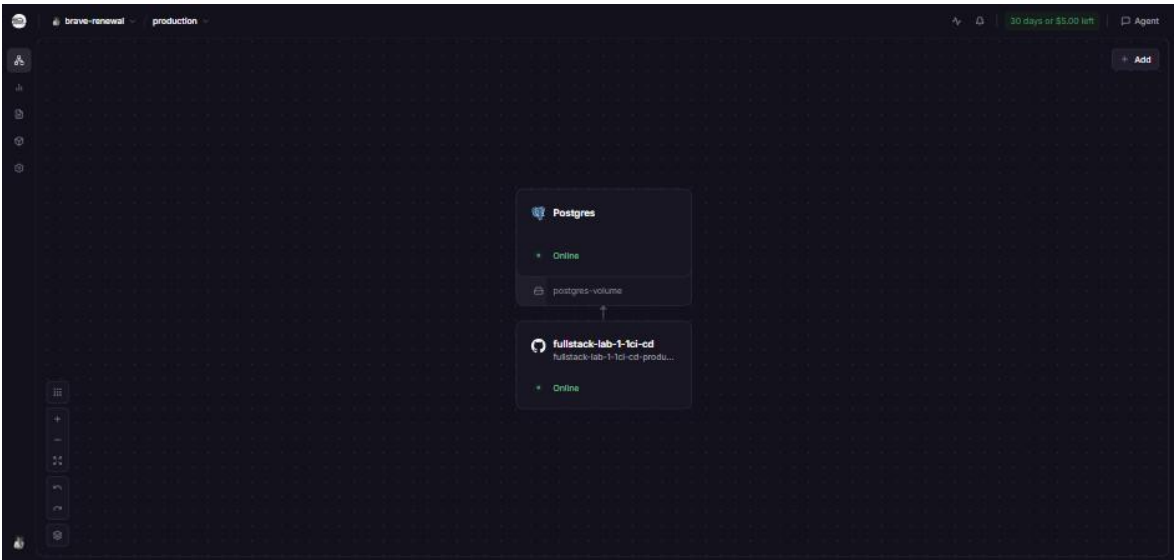


Figura 4. El proyecto con ambos servicios en línea y el dominio público asignado al puerto 4000.

El job de despliegue a staging

```
env:
  SERVICIO_RAILWAY: fullstack-lab-1-1ci-cd

deploy-staging:
  name: Deploy a Staging
  needs: [build, pruebas, migraciones, e2e]
  if: github.ref == 'refs/heads/main' && github.event_name == 'push'
```

```

runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@v4
  - name: Instalar Railway CLI
    run: npm install -g @railway/cli
  - name: Desplegar a Railway (staging)
    run: railway up --service "$SERVICIO_RAILWAY" --ci
  env:
    # Token de proyecto: Railway lo ata a un solo ambiente (staging)
    RAILWAY_TOKEN: ${ secrets.RAILWAY_TOKEN }

```

Guardián	Qué impide
needs: [build, pruebas, migraciones, e2e]	Desplegar algo que no pasó lint, build, pruebas, cobertura, migraciones o E2E. Es el quality gate de la Sesión 4 extendiéndose hasta el despliegue
if: github.ref == 'refs/heads/main' && github.event_name == 'push'	Que un Pull Request abierto dispare un despliegue: solo un merge real lo hace

La prueba de los guardianes

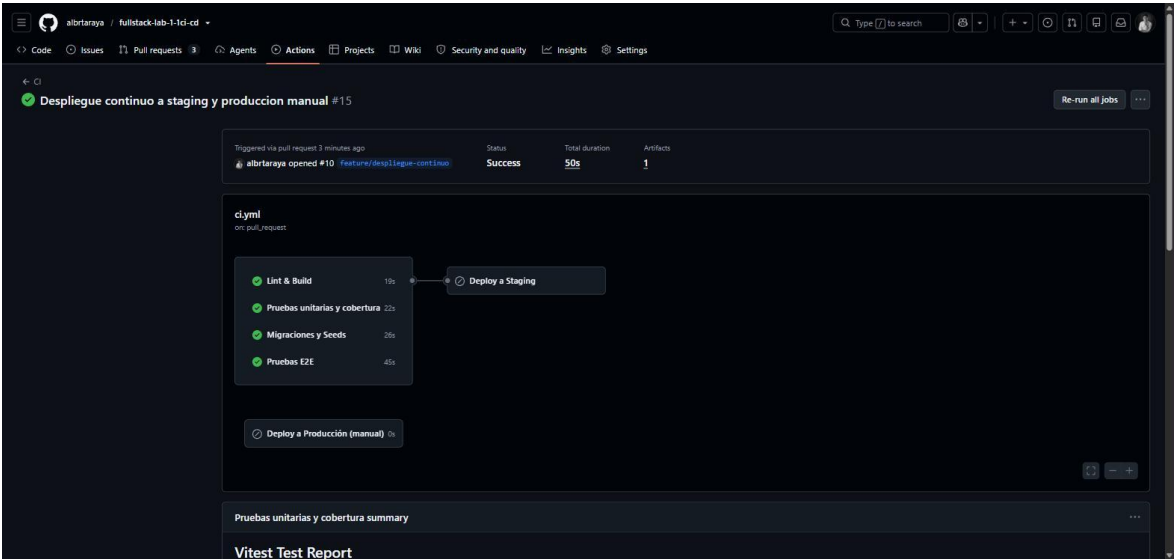
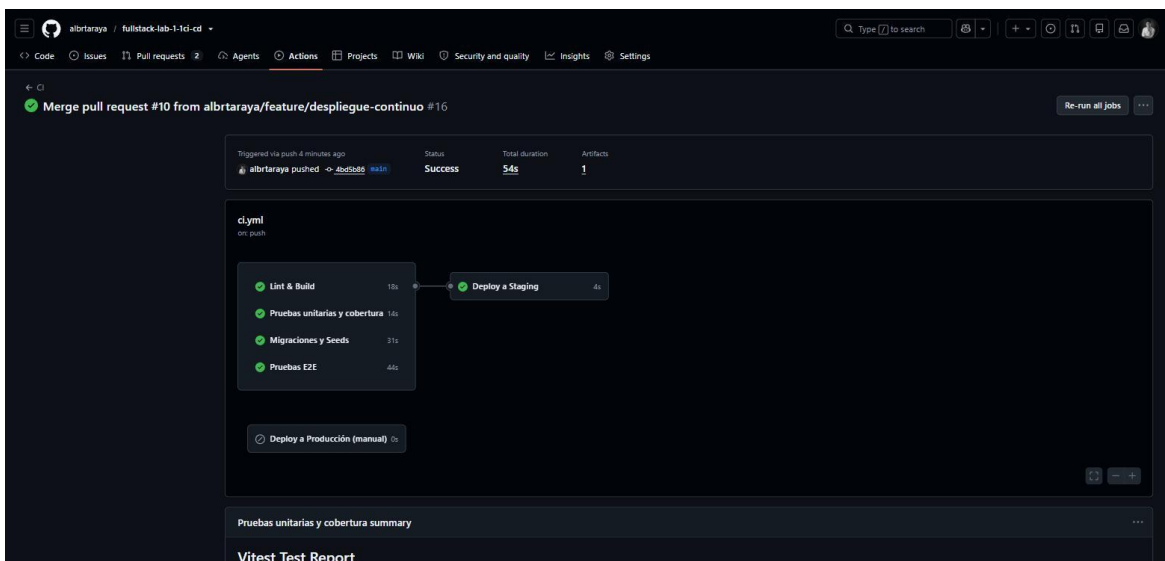
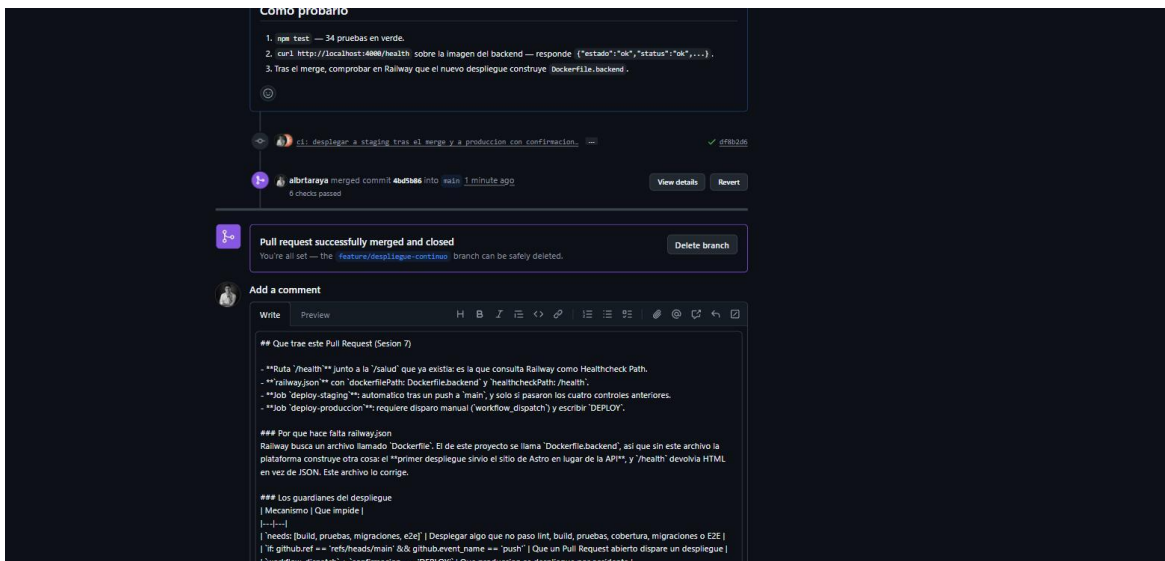


Figura 5. En un Pull Request, los cuatro controles corren y ambos despliegues se omiten.



El token: por qué hicieron falta dos

Railway distingue dos tipos de token. El que se generó primero, desde Account Settings, era de workspace y Railway lo rechazó de las dos maneras posibles:

```
# Como RAILWAY_TOKEN
Invalid RAILWAY_TOKEN. Please check that it is valid and has access to the resource

# Como RAILWAY_API_TOKEN, enlazando antes el proyecto
Unauthorized. Please check that your RAILWAY API TOKEN is valid and has access to the resource
```

Para railway up desde un pipeline se usan tokens de proyecto, que Railway ata a un solo ambiente. Se crearon dos desde Project Settings → Tokens y se guardaron como Secrets:

Secret de GitHub	Token de proyecto para
RAILWAY_TOKEN	ambiente staging
RAILWAY_TOKEN_PRODUCCION	ambiente production

Ventaja de seguridad: con el token de staging es imposible tocar producción. En todos los logs el valor aparece enmascarado como ***.

Nota: el primer token circuló fuera de un canal seguro, así que se revocó. Es el mismo criterio de la Auditoría de Fugas de la Sesión 6: una credencial expuesta se considera comprometida.

Hallazgo: Railway ya no lee railway.json

Con el token funcionando, el log mostró que Railway construía otra cosa:

```
Railpack 0.39.0
└─ Detected Node
└─ Deploying as astro static site
└─ Output directory: dist
```

El panel del servicio explica el motivo: Config as Code quedó obsoleto, y desde el 28 de agosto de 2026 los servicios que nunca lo usaron no pueden activarlo. El railway.json del repositorio se ignora, Railpack detecta Astro y publica el frontend. Eso explica también por qué el primer despliegue de la sesión servía HTML en /health.

La solución es configurar el servicio directamente en el panel, en cada ambiente:

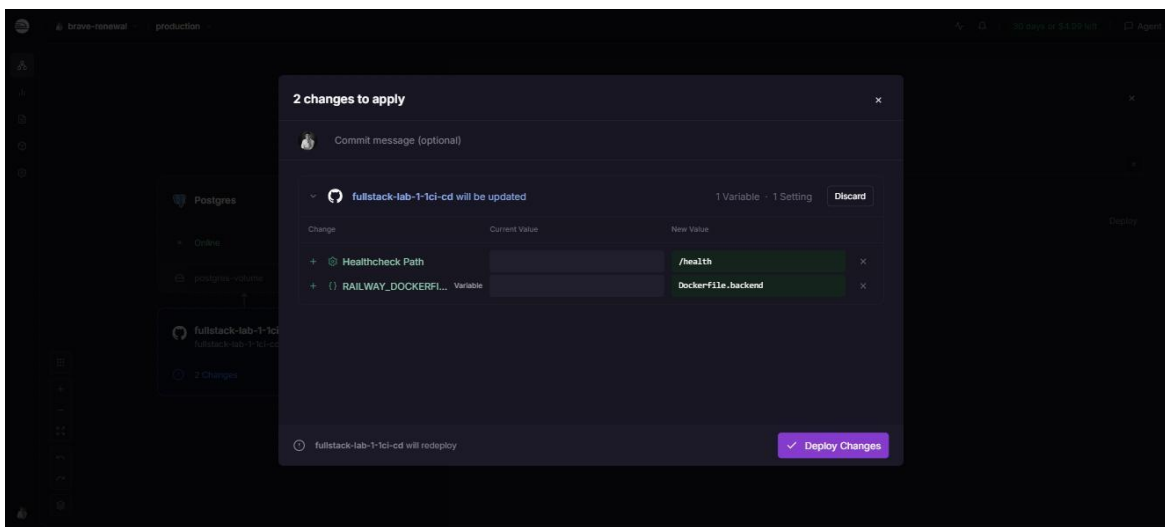


Figura 8. Los dos cambios aplicados al servicio: Healthcheck Path /health y la variable RAILWAY_DOCKERFILE_PATH.

El primer despliegue real a staging

Con todo corregido, el merge del Pull Request #14 disparó el pipeline completo, que terminó en verde en 2 min 53 s. La URL pública de staging responde con el backend real y su base de datos:

```
$ curl https://fullstack-lab-1-1ci-cd-staging.up.railway.app/health
{"estado":"ok","status":"ok","almacen":"postgres"}
```

```
$ curl -X POST ../tarefas -d '{"titulo":"Creada en staging desde el pipeline"}'
{"id":1,"titulo":"Creada en staging desde el pipeline","completada":false}
```

El id 1 confirma que las migraciones corrieron al arrancar: la tabla se creó en la base de staging, que estaba vacía.

Checkpoint del Laboratorio 1

- ✓ El proyecto en Railway está conectado al repositorio y tiene un plugin de PostgreSQL.
- ✓ El token de Railway está guardado como Secret en GitHub, nunca en el código.
- ✓ El job deploy-staging se dispara solo tras un merge a main, y solo si todo lo anterior pasó.
- ✓ La aplicación desplegada responde correctamente en su URL pública.

Laboratorio 2 — Ambientes de staging y producción

Objetivo: separar staging y producción como ambientes independientes, y que producción solo se despliegue por una decisión manual explícita.

Dos ambientes, dos bases de datos

Railway llama production al primer ambiente del proyecto. Se creó staging duplicándolo: Railway copia servicios, variables y configuración, pero cada ambiente recibe su propio plugin de PostgreSQL con su propio volumen.

La separación se comprobó con datos: la tarea creada en staging no existe en producción.

```
staging      1 tareas: ['Creada en staging desde el pipeline']
production   0 tareas: []
```

Al duplicar el ambiente, JWT_SECRET había quedado con el mismo valor en los dos. Se reemplazó en staging por `{{secret(48)}}`, una función de Railway que genera el valor en el servidor. Los dos ambientes quedaron con secretos distintos sin que nadie tuviera que escribir ni copiar el valor.

El disparador manual

```
on:
  workflow_dispatch:
    inputs:
      confirmacion:
        description: 'Escribe DEPLOY para confirmar el despliegue a produccion'
        required: true

  deploy-produccion:
    if: github.event_name == 'workflow_dispatch'
      && github.event.inputs.confirmacion == 'DEPLOY'
    ...
    run: railway up --service "$SERVICIO_RAILWAY" --ci
    env:
      RAILWAY_TOKEN: ${{ secrets.RAILWAY_TOKEN_PRODUCCION }}
```

Despliegue manual a producción

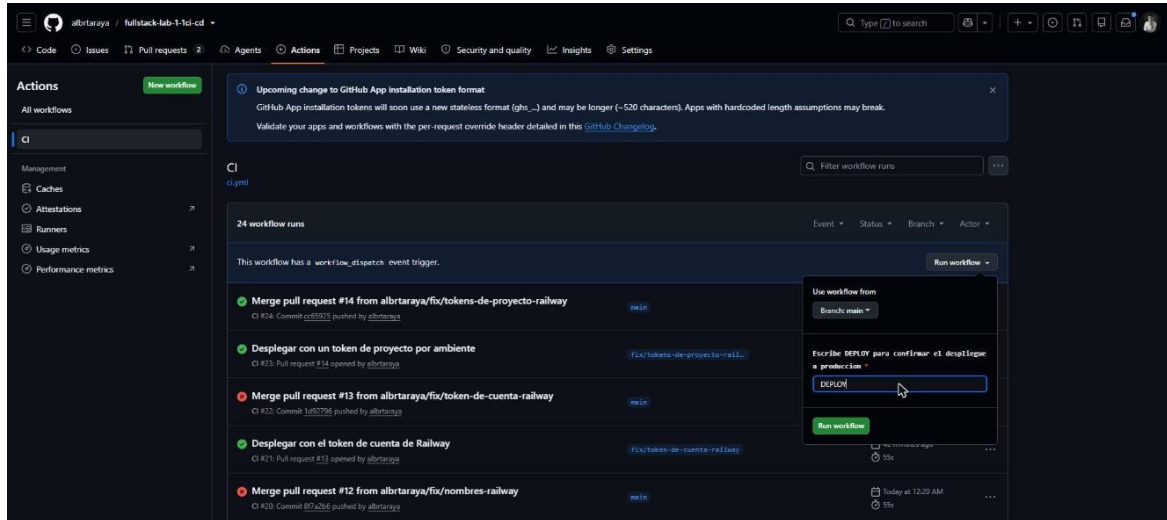


Figura 9. Actions → CI → Run workflow, con DEPLOY escrito a mano en el campo de confirmación.

El run #25 se ejecutó con Deploy a Staging omitido y Deploy a Producción en marcha. El log del job termina así:

```
Starting Healthcheck
[1/1] Healthcheck succeeded!
Deploy complete

$ curl https://fullstack-lab-1-1ci-cd-production.up.railway.app/health
{"estado":"ok","status":"ok","almacen":"postgres"}
```

Checkpoint del Laboratorio 2

- ✓ Staging y producción son ambientes separados en Railway, con sus propias variables y base de datos.
- ✓ El despliegue a producción requiere workflow_dispatch con confirmación explícita.
- ✓ Un push normal a main nunca dispara el despliegue a producción, solo a staging (Figura 7).

Laboratorio 3 — Simulacro de Caos

Objetivo: vivir el ciclo completo de un incidente en staging — provocarlo, diagnosticarlo bajo presión de tiempo y revertirlo — antes de corregir el problema real con calma.

Ronda 1 — Provocar el desastre

Se eligió la opción A de la guía: una variable de entorno con el nombre incorrecto en el arranque del servidor.

```
// src/server/index.js
// Validacion de arranque: sin base de datos no tiene sentido levantar la API.
if (!process.env.DATABASE_URL) {
  throw new Error('DATABASE_URL no esta definida: la API no puede arrancar sin base de datos');
}
```

Por qué no la opción B: un throw suelto antes de app.listen() deja código inalcanzable y ESLint lo frena con la regla no-unreachable. El quality gate lo habría detenido antes de llegar a staging, que es una buena noticia, pero no permitía hacer el simulacro.

Como anticipaba la guía, el Pull Request #15 pasó los cuatro controles: ninguna prueba carga index.js y ESLint no puede saber que el nombre de la variable está mal. Tras el merge, el despliegue a staging falló:

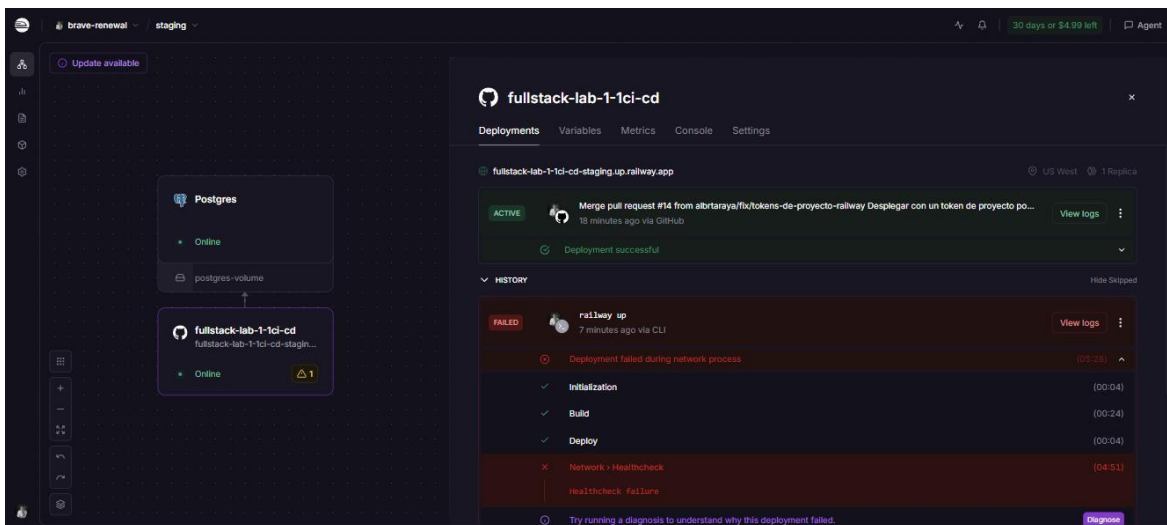


Figura 10. El despliegue de railway up queda en FAILED: pasa Build y Deploy, pero Network > Healthcheck falla tras 4:51. El ACTIVE sigue siendo el despliegue sano.

Mientras duró el incidente se consultó la URL pública cada 30 segundos:

05:35:03	pipeline=queued	staging/health=200
05:37:11	pipeline=in_progress	staging/health=200
05:39:51	pipeline=in_progress	staging/health=200
05:42:43	pipeline=failure	staging/health=200

Lo que enseña este resultado: staging nunca dejó de responder. Railway no promueve un despliegue cuyo healthcheck falla, así que el anterior siguió atendiendo. La ruta /health configurada en el Laboratorio 1 convirtió una caída segura en un despliegue fallido sin impacto para los usuarios.

Ronda 2 — Diagnóstico bajo presión

Cronómetro desde las 05:42:43, cuando el pipeline marcó el fallo. Ruta seguida: Railway → staging → Deployments → despliegue FAILED → Deploy Logs.

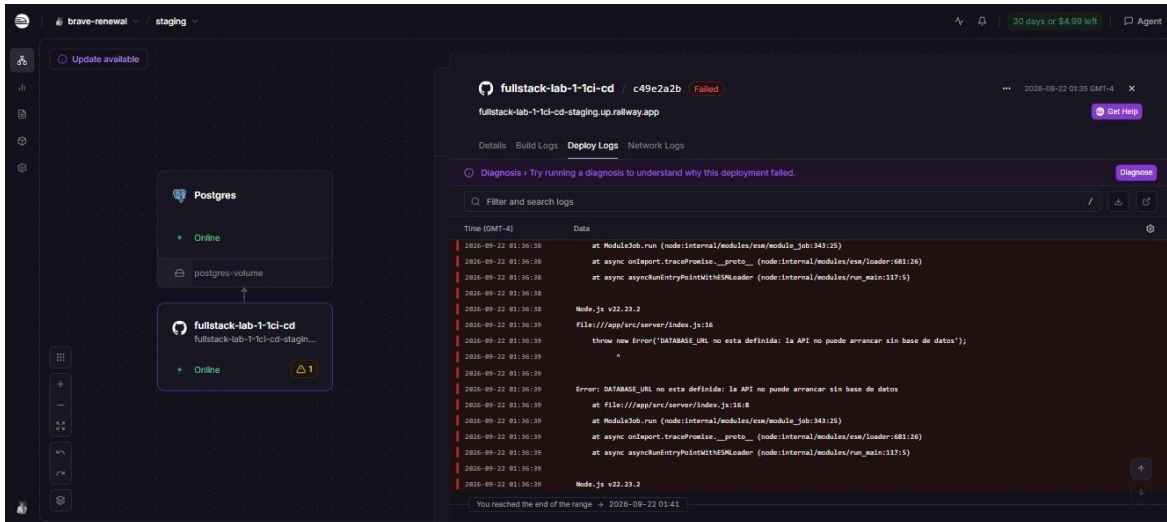


Figura 11. Los Deploy Logs muestran la excepción, el archivo y la línea exacta, repetidos por cada reinicio del contenedor.

Causa, en una frase: el servidor muere en `src/server/index.js:16` porque la validación de arranque consulta `process.env.DATABASE_URL` (con una L de más), una variable que no existe. El mensaje de error engaña: nombra `DATABASE_URL`, que sí está definida en Railway.

Tiempo de diagnóstico: unos 2 minutos, dentro del límite de 10.

Ronda 3 — Rollback y verificación

En el historial de Deployments, sobre el último despliegue sano: menú : → Redeploy → confirmar.

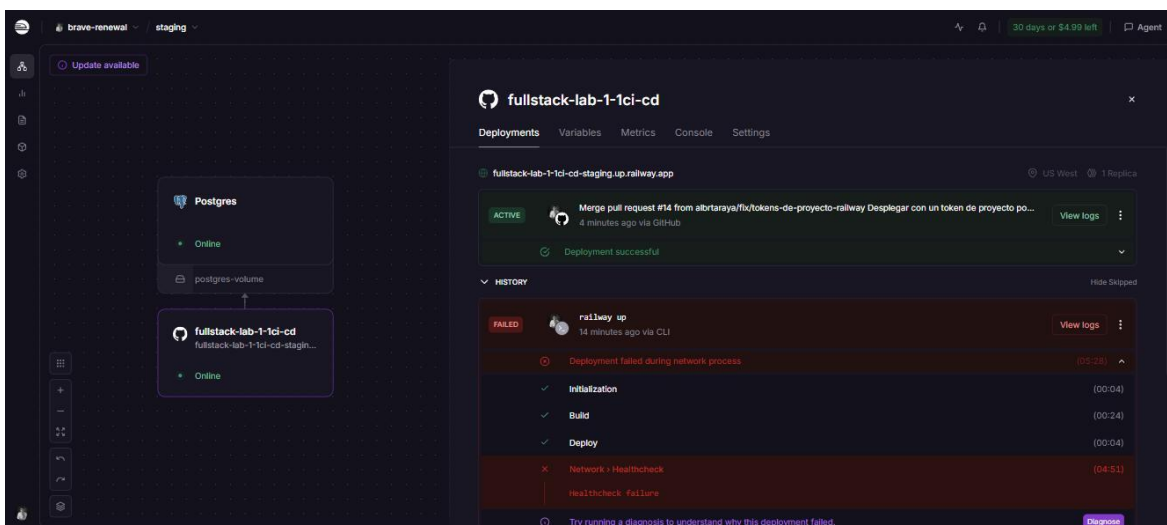


Figura 12. Después del rollback: el despliegue ACTIVE es la versión sana redespuesta y el servicio figura Online, sin el aviso de fallo.

```
rollback lanzado 05:45:51 UTC
05:45:52 {"estado":"ok","status":"ok","almacen":"postgres"}
05:47:57 {"estado":"ok","status":"ok","almacen":"postgres"}
05:49:41 {"estado":"ok","status":"ok","almacen":"postgres"}
```

La corrección, por el flujo normal

Recién con el servicio estable se corrigió el error real, en su propia rama y con su propio Pull Request (#16):

```
-if (!process.env.DATABASE_URL) {
+if (!process.env.DATABASE_URL) {
```

Tras el merge, el pipeline de main terminó en verde en 1 min 55 s. Esta vez el despliegue a staging pasó el healthcheck y quedó ACTIVE. El historial de Railway resume el incidente completo, de abajo hacia arriba: el despliegue sano, el FAILED del simulacro, el redeploy del rollback y la corrección.

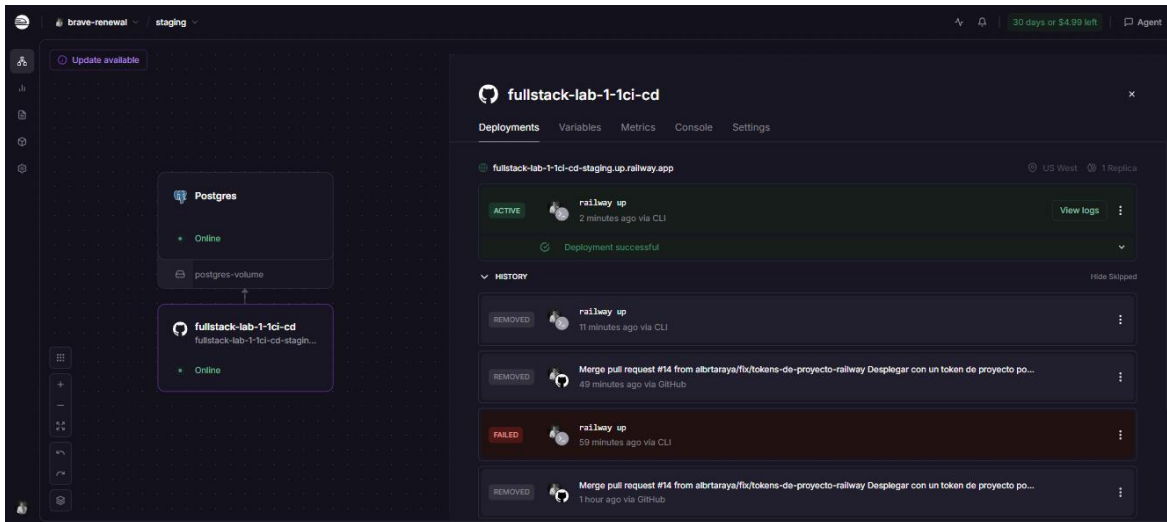


Figura 13. Historial de staging: el FAILED del simulacro, el redeploy del rollback y, arriba, los despliegues posteriores ya sanos.

Checkpoint del Laboratorio 3

- ✓ El servicio tiene una ruta /health configurada como Healthcheck Path en Railway.
- ✓ Se provocó un despliegue defectuoso real y se diagnosticó con los logs y el healthcheck.
- ✓ Se ejecutó un rollback exitoso y luego se corrigió el error real por el flujo normal.

Insignias

Insignia	Puntos	Evidencia
👨‍⚕️ Médico de Servicios	10	Causa identificada solo con logs y healthcheck (Figura 11)
🦸‍♂️ Héroe del Rollback	15	Rollback exitoso, con el servicio sano de nuevo (Figura 12)

Entregable final de la Sesión 7

Requisito	Estado
El pipeline despliega a staging tras cada merge, solo si todos los checks pasaron	Cumplido
Staging y producción son ambientes completamente separados	Cumplido
El despliegue a producción requiere disparo manual explícito	Cumplido
El servicio tiene una ruta /health configurada como healthcheck	Cumplido
Se provocó un despliegue defectuoso y se diagnosticó con logs	Cumplido
Se ejecutó un rollback y luego se corrigió el error real	Cumplido

Lo aprendido

1. Un token no es solo un valor secreto: también tiene un alcance. El de workspace no despliega; el de proyecto sí, y limitado a un ambiente.
2. La documentación y la plataforma cambian. railway.json era la forma recomendada y hoy se ignora en servicios nuevos. Leer el log («Deploying as astro static site») evitó perseguir el error equivocado.
3. Un healthcheck bien configurado es la primera línea del rollback: impidió que el despliegue roto llegara a atender a nadie.
4. Pasar el quality gate no garantiza que el servicio arranque. El error del simulacro vivía en el único archivo que ninguna prueba ejecuta.

Anexo A — Índice de evidencias

Archivo	Contenido
01-railway-cuenta-sin-proyectos.jpg	La cuenta antes de empezar
02-postgres-aprovisionado.jpg	PostgreSQL con su volumen
03-variables-del-servicio.jpg	Variables enlazadas al plugin
04-proyecto-con-dominio-publico.jpg	Servicios en línea y dominio
05-despliegue-activo-railway.jpg	Primer despliegue, que servía Astro
06-pr-ambos-deploy-omitidos.jpg	Run #15: en un PR se omiten ambos deploys
07-pr10-merged.jpg	Pull Request #10 integrado
08-push-staging-corre-produccion-no.jpg	Run #16: staging sí, producción no
09-configuracion-servicio-produccion.jpg	Healthcheck y Dockerfile en el panel
10-run-workflow-deploy.jpg	Disparo manual con DEPLOY
11-staging-healthcheck-fallido.jpg	El despliegue roto, con healthcheck en rojo
12-logs-del-error.jpg	Deploy Logs con la causa del fallo
13-rollback-staging-sano.jpg	Staging sano después del rollback
15-historial-del-incidente.jpg	Despliegue de la corrección, sano

Anexo B — Pull Requests de la sesión

PR	Título	Resultado
#10	Desplegar a staging tras el merge y a producción con confirmación	Integrado
#12	Invocar el servicio de Railway por su nombre real	Integrado
#13	Desplegar con el token de cuenta de Railway	Integrado, luego reemplazado
#14	Desplegar con un token de proyecto por ambiente	Integrado
#15	Simulacro de caos: validar la base de datos al arrancar	Integrado (error intencional)
#16	Corregir el nombre de DATABASE_URL en el arranque	Integrado